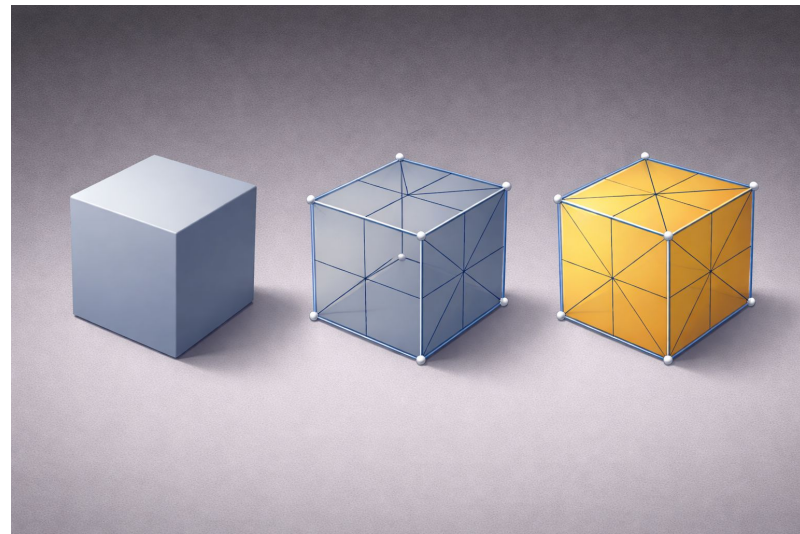


Técnicas poligonais para controle de desempenho

Onde paramos na aula anterior

- Objetos são representados como **malhas poligonais**
- Malhas são compostas por vértices, arestas e faces
- No fim do pipeline, tudo vira triângulo
- A GPU trabalha com triângulos como primitiva básica

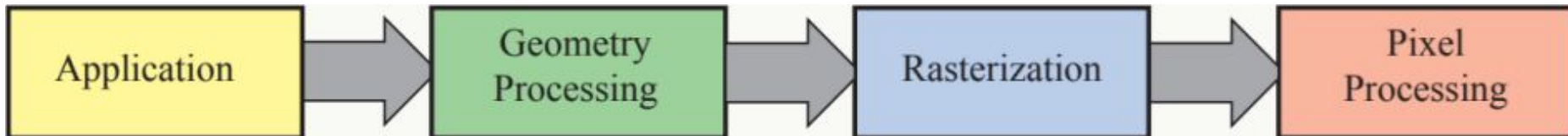


Do modelo ao frame na tela

Fluxo simplificado em tempo real:

- Aplicação envia malha para a GPU
- Vertex shader processa vértices
- Triângulos são montados
- Rasterização gera fragmentos
- Fragment shader calcula cor final

Ponto importante: cada triângulo atravessa esse **fluxo inteiro**.



A pergunta que guia esta aula

Se tudo vira **triângulo**, então:

- Quantos triângulos posso usar?
- Onde devo colocar mais detalhe?
- Quando estou desperdiçando desempenho?
- Como reduzir custo sem destruir qualidade?

Ideia central da aula:

- Geometria não é só forma.
- É decisão de desempenho em tempo real.

Vertex Processing escala com triângulos

Cada triângulo possui **3 vértices**.

Para cada vértice:

- **Vertex shader** é executado
- **Transformações** são aplicadas
- Dados são preparados para rasterização

Se você passa de:

- 10k → 100k triângulos

Você **multiplicou por 10**:

- Execuções do vertex shader
- Leitura de atributos
- Processamento geométrico



Aqui o gargalo é geométrico.

Rasterização não depende só da quantidade

Depois do estágio geométrico, os triângulos são rasterizados.

O custo agora depende de:

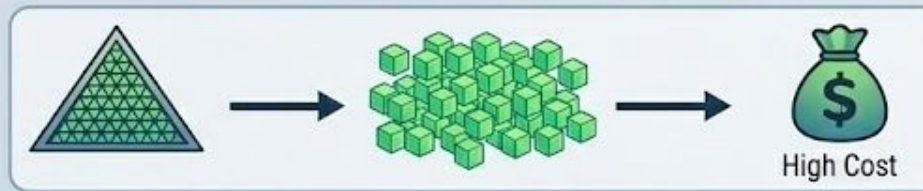
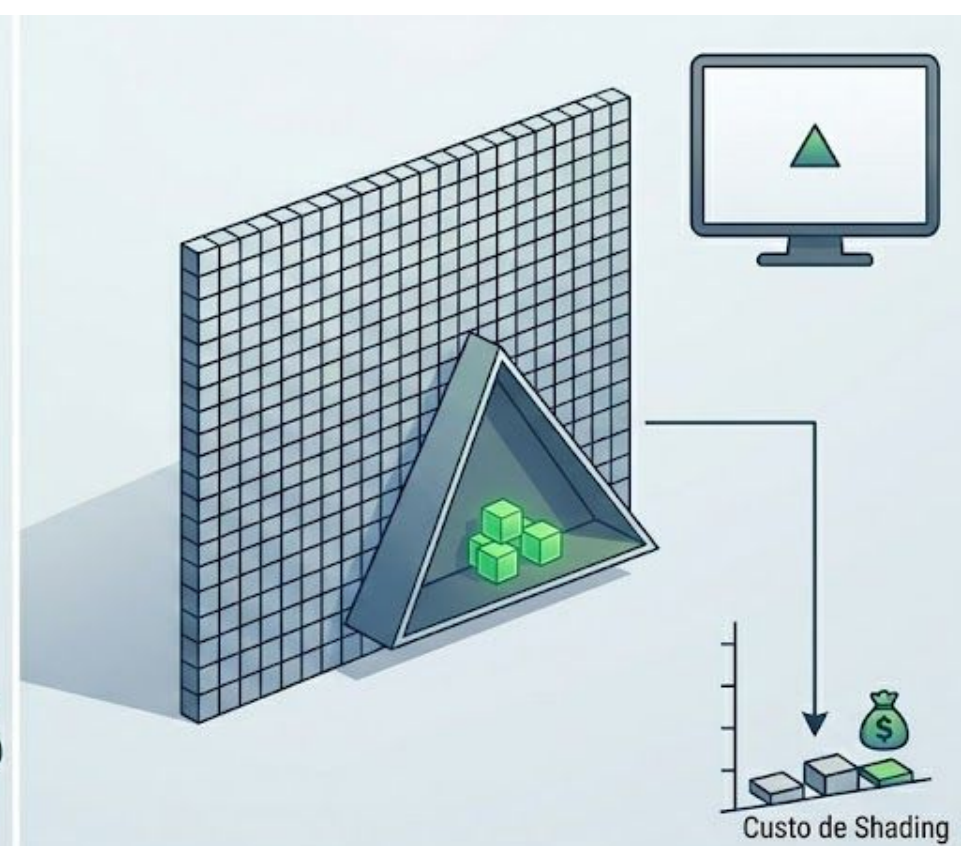
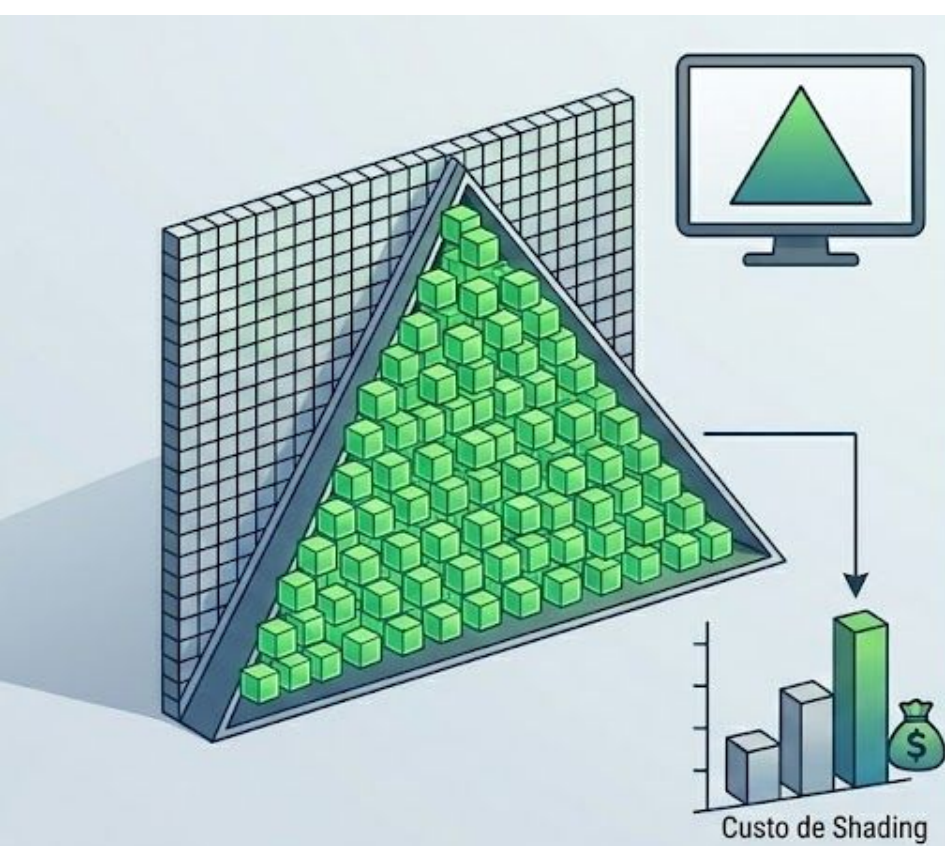
- **Tamanho** do triângulo na tela
- **Quantidade** de pixels cobertos

Dois modelos com o mesmo número de triângulos podem ter custos muito diferentes se:

- Um ocupa a tela inteira
- Outro está distante e pequeno

Triângulo grande → mais fragmentos

Triângulo pequeno → menos fragmentos



Overdraw: o custo invisível

Overdraw ocorre quando um pixel é processado várias vezes antes da versão final.

Causas comuns:

- Geometria sobreposta
- Transparência
- Cenas densas

Mais triângulos sobrepostos = **mais fragment shaders** executados para o mesmo pixel.

- Isso vira **gargalo de fragmento**.

Gargalo geométrico vs gargalo de fragmento

Existem dois cenários típicos:

- Cenário A - Gargalo geométrico
 - Muitos triângulos pequenos
 - Modelos extremamente detalhados
 - Vertex shader pesado
- Cenário B - Gargalo de fragmento
 - Triângulos grandes
 - Tela quase toda coberta
 - Fragment shader complexo

diagnóstico errado

=

otimização errada

Budgets reais em jogos

Engines não trabalham com “quantos triângulos eu quiser”, elas trabalham com **orçamento**.

Exemplo típico (varia por plataforma):

- Personagem principal: limite específico
- NPCs: orçamento menor
- Cenário distante: orçamento reduzido
- Objetos pequenos: simplificados

Distribuição de triângulos é **decisão estratégica** → Não é artística. **É técnica.**

Exemplo 1 - Modelo base com múltiplas versões de malha:

Modelo com níveis de detalhe (LOD):

- LOD A $\approx$ 5k triângulos
- LOD B $\approx$ 50k triângulos
- LOD C $\approx$ 500k triângulos

Controles:

1. Trocar LOD
2. Wireframe
3. Zoom da câmera
4. Shader simples $\leftrightarrow$ pesado
5. Overdraw (heatmap)

HUD na tela:

- FPS
- Triângulos ativos
- Tempo de GPU

O que observar:

- FPS $\downarrow$ com mais triângulos
- Distância altera o custo
- Gargalo geométrico vs fragmento
- Microtriângulos não mudam silhueta

Mesmo código, variando apenas:

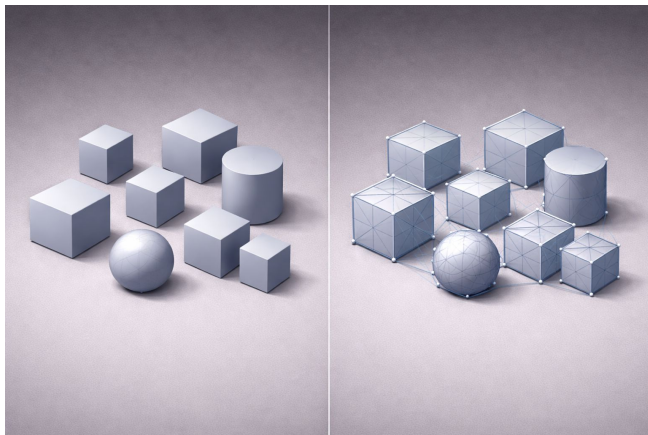
- – N° de triângulos
- – Tamanho na tela
- – Shader

Técnicas de controle estrutural da malha

Até agora falamos de **quantidade**, mas agora a pergunta muda:

- Como esses triângulos estão distribuídos?
- Como estão enviados para a GPU?
- Quantas chamadas de desenho estou fazendo?

Desempenho não depende só da malha, depende também estrutura da cena.



Consolidação de malhas (Mesh Consolidation)

Problema comum:

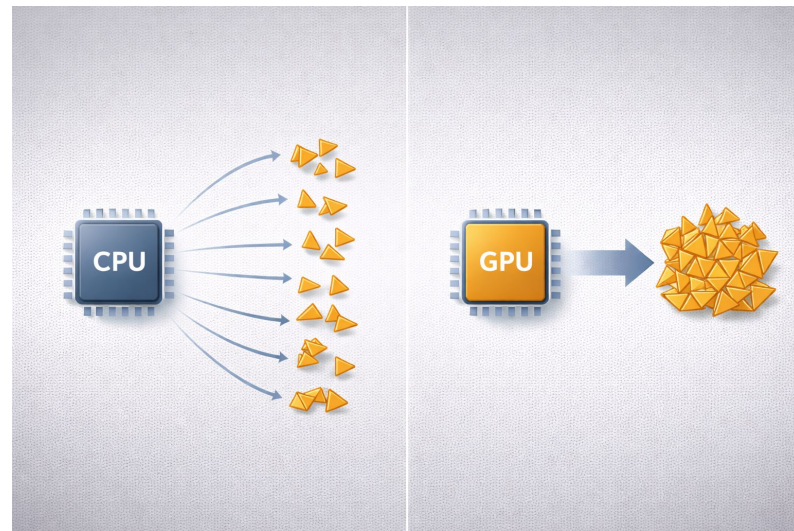
- 100 objetos pequenos,
- Cada um com poucos triângulos
- 100 draw calls

Mesmo com poucos triângulos, o custo pode ser alto, pois cada draw call tem custo de CPU

Há custos de:

- Validação no driver
- Troca de estado na GPU

Mensagem central: Poucos triângulos + muitas draw calls = **gargalo de CPU**.



Batching: juntando para ganhar eficiência

Batching = agrupar múltiplas malhas em uma só.

Benefícios:

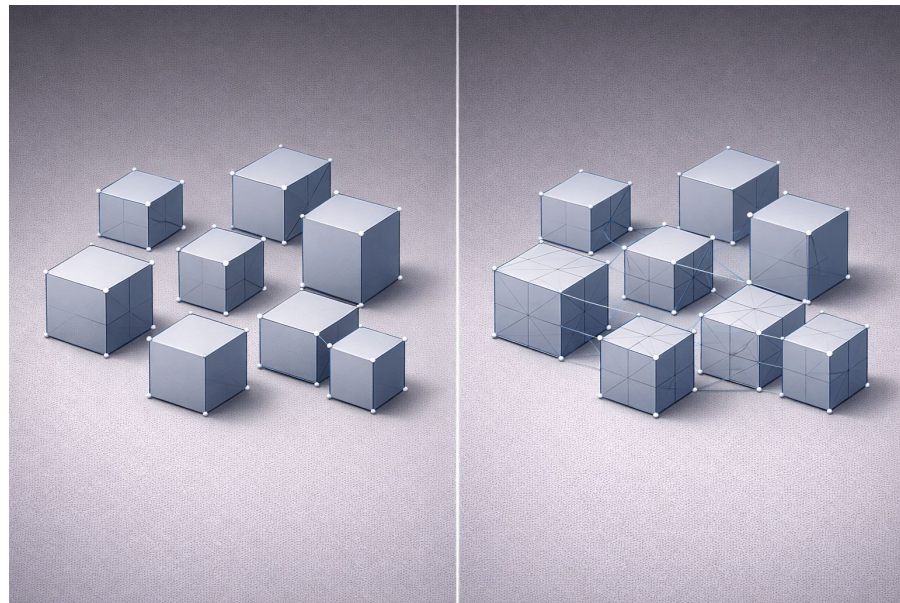
- Menos draw calls
- Melhor uso de cache
- Menos troca de estado

Funciona melhor quando:

- Mesmos materiais
- Mesmo shader
- Objetos estáticos

Cuidados:

- Pode aumentar uso de memória
- Pode dificultar culling fino



Quando NÃO consolidar

Nem sempre consolidar é ideal. Evite quando:

- Objetos são animados independentemente
- Precisam de culling individual
- Têm materiais diferentes
- Precisam ser removidos dinamicamente

Consolidação excessiva pode:

- Aumentar overdraw
- Reduzir eficiência de frustum culling

Otimização errada também custa.

Simplificação geométrica

Agora voltamos para o interior da malha.

Simplificar é:

- Reduzir número de triângulos
- Preservar forma perceptível
- Manter silhueta

Técnica clássica: **Edge collapse**

Objetivo: Remover triângulos onde o erro visual é mínimo.



Silhueta é mais importante que detalhe interno

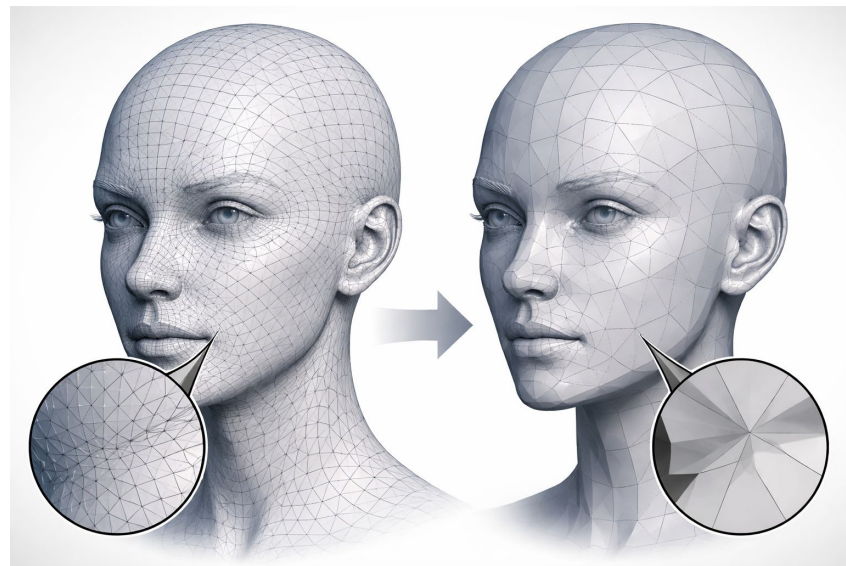
O olho percebe principalmente:

- Contorno externo
- Mudanças bruscas de forma

Detalhe interno fino:

- Muitas vezes invisível em movimento
- Pode ser substituído por normal map

Erro comum: Distribuir triângulos uniformemente em vez de concentrar onde a forma muda.

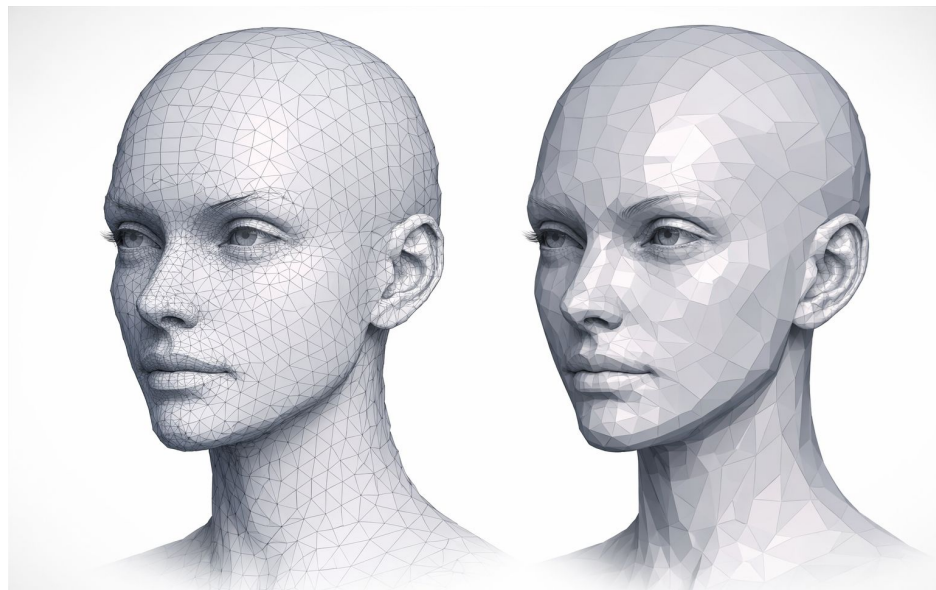


Erros comuns na simplificação

Problemas típicos:

- Perda da silhueta
- Artefatos em iluminação
- Normais inconsistentes
- Triângulos longos e degenerados

Simplificar não é só remover → É remover mantendo coerência estrutural.



Uso real em pipelines

Na prática:

- Ferramentas geram múltiplas versões simplificadas
- Engines importam já com LODs
- Simplificação é feita offline

Motivo:

- Processar simplificação em tempo real é caro.
- O ideal é **pré-processar**.

Nível de Detalhe (LOD) e adaptação dinâmica

Até agora:

- Ou temos malha densa
- Ou temos malha simplificada

Mas a **câmera se move**, a **distância muda**, e a **importância visual muda**.

Pergunta: Por que usar 500k triângulos para um objeto que ocupa 20 pixels?

O que é LOD (Level of Detail)

LOD é usar versões diferentes da mesma malha:

- Alta resolução (perto)
- Média resolução
- Baixa resolução (longe)

A troca depende normalmente de:

- Distância da câmera
- Tamanho projetado na tela

Objetivo:

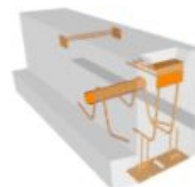
- Manter qualidade perceptível
- reduzindo custo desnecessário.



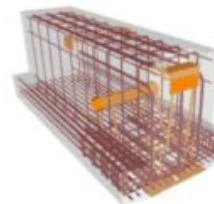
LOD 200



LOD 300



LOD 350



LOD 400

LOD discreto

LOD discreto:

- Troca abrupta entre versões
- Simples de implementar
- Muito usado em engines

Problema: Pode gerar “**popping**” (mudança visível súbita)

Exemplos:

https://www.reddit.com/r/starcitizen/comments/1n392k3/this_lod_method_is_uhh_in_need_of_some_work/

https://www.reddit.com/r/FalloutMods/comments/p7xtrw/fo4_is_this_amount_of_lod_texture_popping_normal/

LOD contínuo

LOD contínuo:

- Transição suave
- Interpolação entre níveis
- Pode usar morphing geométrico

Vantagem: menos popping

Custo:

- Mais complexo de implementar
- Mais memória

LOD discreto vs continuo



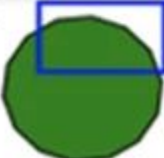
Figure 1: Left: Discrete LOD structure with sudden jumps in density and popping artifacts under motion. Middle: Screenshot of our method in virtual reality. No visible seams across different levels of detail due to a continuous reduction in density, and also continuously reduced density towards the periphery to reduce geometric complexity where less is needed. Endeavor point cloud courtesy of NVIDIA [4]. Right: Continuous transition from one LOD to another.

Screen-Space Error

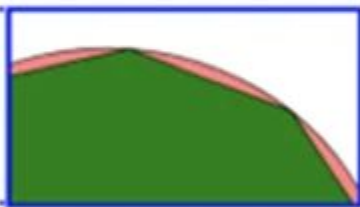
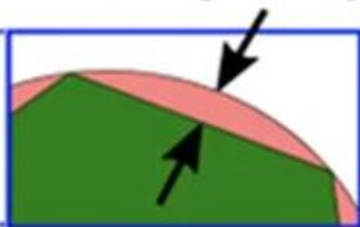
Decisão moderna de LOD: basear não apenas na distância, mas no erro visual projetado na tela.

Ideia: Se o erro não é perceptível em pixels, não vale manter alta densidade.

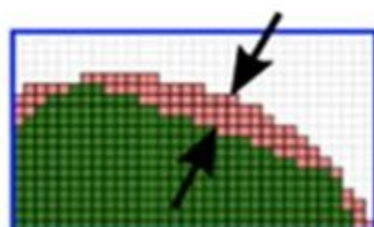
Isso conecta geometria com percepção visual.

tile	geometricError: 61
content:	
children[0]:	geometricError: 28
content:	
children[0]:	geometricError: 12
content:	

The geometric error is measured in meters, comparing the simplified geometry to the real geometry:



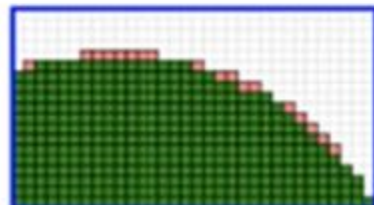
The screen space error (SSE) is measured in pixels:



SSE:
about
4 pixels



SSE:
about
2 pixels



SSE:
about
1 pixel

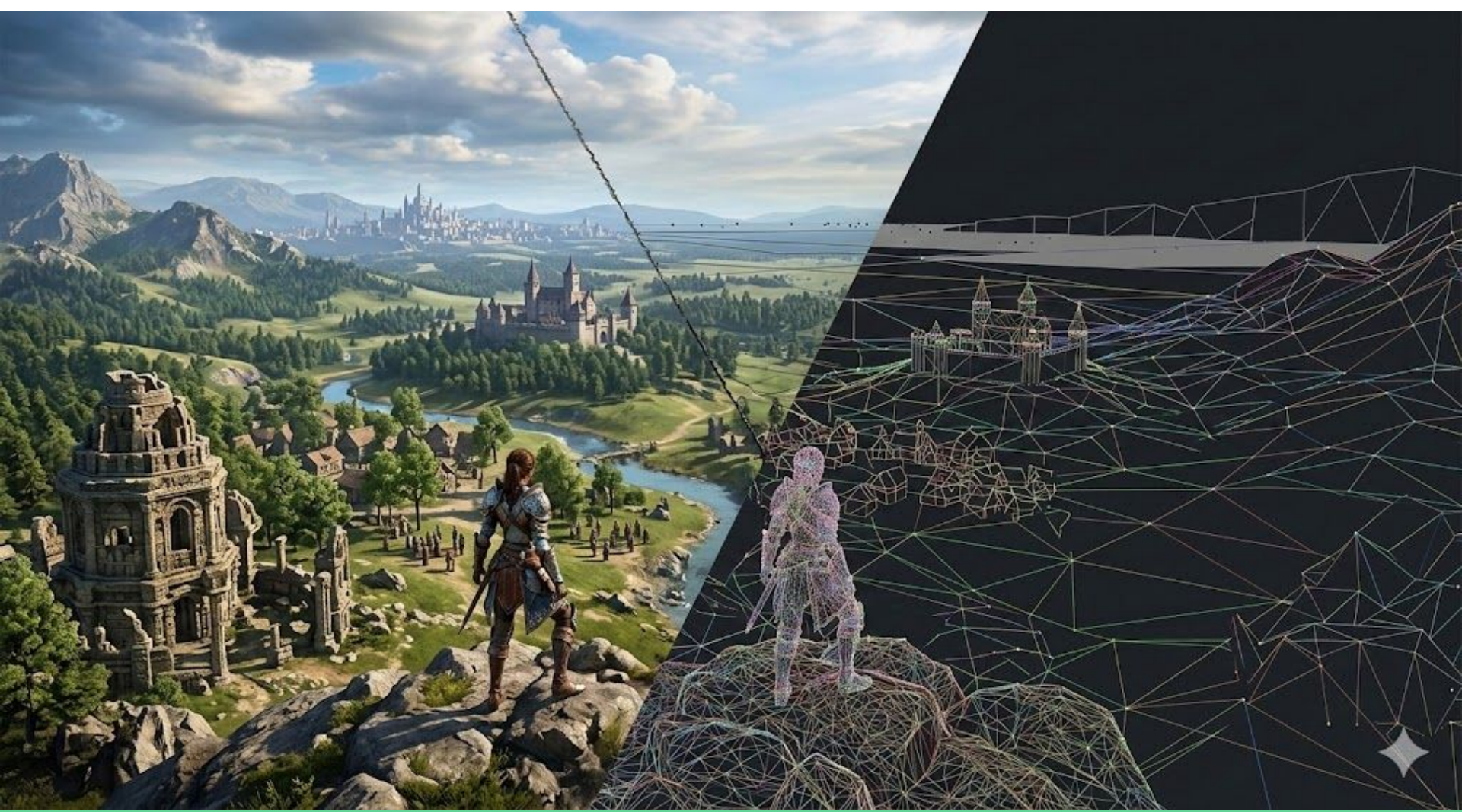
LOD em jogos reais

Uso típico:

- Personagens principais: LOD controlado
- NPCs distantes: LOD agressivo
- Cenários grandes: múltiplos níveis

Open world depende fortemente de LOD.

Sem LOD é quase impossível escalar cena grande.



Normal Map



Displacement



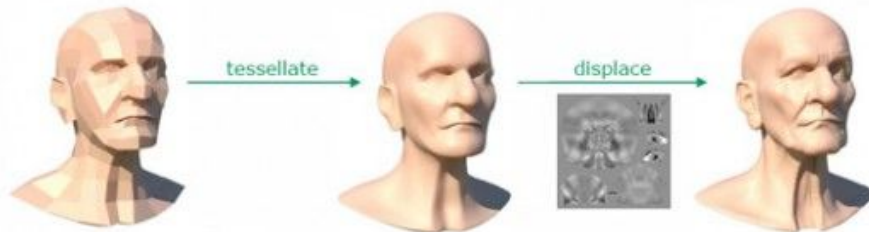
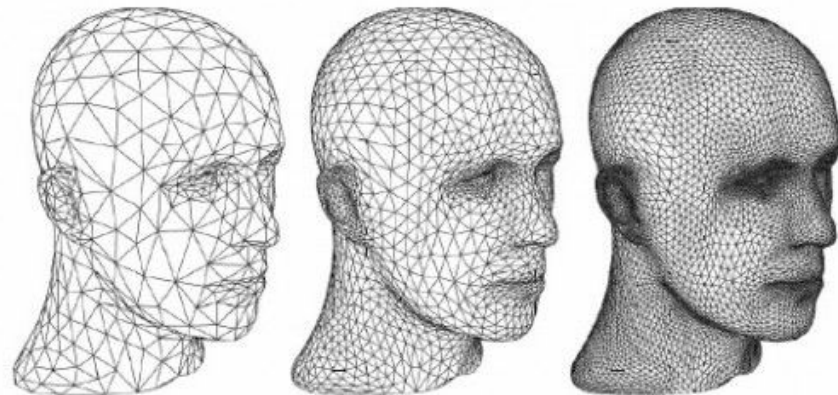
Tessellation e decisões de design

Tessellation é subdividir triângulos em tempo real na GPU.

Fluxo simplificado:

- Malha base simples
- Tessellation shader subdivide
- Opcionalmente aplica displacement

Resultado: **mais triângulos** gerados dinamicamente.



Por que usar Tessellation?

Vantagens:

- Superfícies curvas mais suaves
- Terrenos com detalhe adaptativo
- Displacement real (não apenas normal map)
- Detalhe concentrado onde necessário

Permite a utilização de **malhas mais leves em memória** e **geração de detalhes** conforme necessidade.



Quando evitar Tessellation

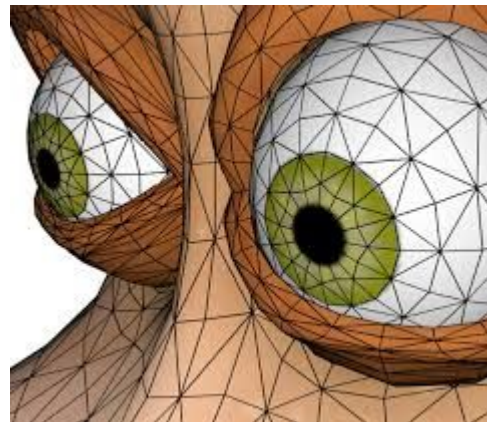
Tessellation pode gerar:

- Explosão de triângulos
- Microtriângulos
- Gargalo geométrico
- Overhead desnecessário

Evitar quando:

- Objeto é pequeno na tela
- Já possui alta densidade
- O detalhe pode ser resolvido com normal map

Mais geometria não significa melhor resultado sempre.



Displacement vs Normal Map

Displacement:

- Altera geometria real
- Afeta silhueta
- Aumenta número de triângulos

Normal map:

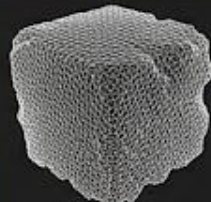
- Simula detalhe
- Não altera silhueta
- Muito mais barato

Decisão depende se o detalhe altera a silhueta.



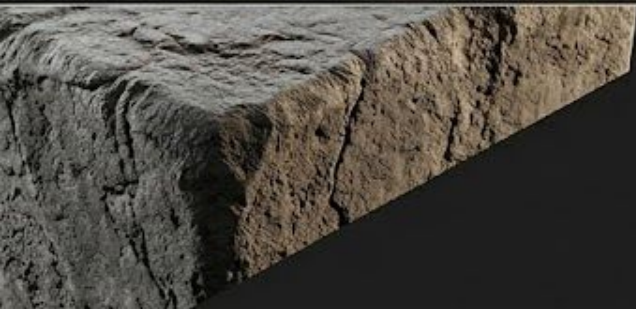
Normal Map

(Simulated Depth, Unchanged Silhouette)



Displacement Map

(Real Geometry, Affected Silhouette)



Cost



Cost

Distribuição inteligente de polígonos

Princípio central: Coloque triângulos onde a forma muda.

Evite:

- Densidade uniforme
- Detalhe interno invisível
- Microtriângulos desnecessários

Concentre:

- Na silhueta
- Em áreas de alta curvatura
- Onde o jogador realmente olha

Decisão de design orientada a desempenho

Perguntas práticas:

- O detalhe altera a silhueta?
- O objeto ocupa muitos pixels?
- Ele é foco da cena?
- Está próximo da câmera?

Se a resposta for não, provavelmente não precisa de mais geometria.

Geometria é recurso limitado, portanto, **distribua como orçamento.**

Encerramento

Geometria é decisão técnica. Você controla:

- Quantidade
- Organização
- Adaptação
- Geração dinâmica

Sempre conectando:

- Decisão geométrica → impacto no pipeline → impacto visual → impacto no desempenho.